03 — Preprocesamiento

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook define, valida y analiza el efecto de la cadena de preprocesamiento aplicada a cada radiografía antes de extraer características.

Cadena de preprocesamiento

```
imagen original → escala de grises → resize 256×256 → Bilateral Filter → CLAHE
```

Justificación de cada paso

Paso	Propósito
Escala de grises	Las radiografías de rayos X son intrínsecamente imágenes de intensidad
Resize 256×256	Normaliza las dimensiones heterogéneas del dataset para cálculos uniformes
Bilateral Filter	Reduce ruido preservando los bordes de estructuras anatómicas
CLAHE	Realza el contraste local para revelar estructuras en zonas poco contrastadas

0. Configuración del entorno

In [1]:
```python
# Se configura el path del proyecto y se importan las funciones del framework.

import sys, os, warnings
warnings.filterwarnings('ignore')

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import numpy as np
import pandas as pd
from pathlib import Path

from src.data import download_dataset, get_data_root, build_base_dataframe, PALETTE

# Configuración central del pipeline: definida una sola vez en src/config.py
from src.config import N_INTENSIDAD, N_SEGUIMIENTO
from src.preprocessing import (
    preprocess,           # Pipeline completo: resize + BF + CLAHE
    preprocess_steps,     # Retorna cada paso intermedio por separado
    IMG_SIZE, BF_D, BF_SIGMA_COLOR, BF_SIGMA_SPACE,
    CLAHE_CLIP_LIMIT, CLAHE_TILE_GRID,
)
from src.visualization import (
    set_publication_style,
    plot_preprocessing_steps,
    plot_histogram_comparison,
    plot_intensity_histograms,
    compute_intensity_stats,
    plot_intensity_metrics,
    build_preprocessing_stats_table,
)

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga del DataFrame base

In [2]:
```
# Se carga el DataFrame base desde data/df_base.csv.
# Si aún no existe, ejecutar primero el notebook 01 para generarlo.

CSV_PATH = Path('..') / 'data' / 'df_base.csv'

if CSV_PATH.exists():
    df_base = pd.read_csv(CSV_PATH)
    print(f'DataFrame cargado desde: {CSV_PATH.resolve()}')
    print(f'Total imágenes: {len(df_base)}')
else:
    print(f'[ERROR] No se encontró: {CSV_PATH.resolve()}')
    print('Ejecuta primero el notebook 01 para generar df_base.csv')

print()
print(df_base['class_name'].value_counts())
```

```
DataFrame cargado desde: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\df_base.csv
Total imágenes: 8400

class_name
health    3800
sick      3800
tb         800
Name: count, dtype: int64
```

2. Parámetros del pipeline de preprocesamiento

In [3]:
```
# Se muestran los parámetros configurados en `src/preprocessing.py`.
# Modificar los valores en ese módulo afecta todos los notebooks del pipeline
# de forma consistente.
# Referencia BF: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html
# Referencia CLAHE: https://docs.opencv.org/4.x/d6/db6/classcv_1_1CLAHE.html

import importlib
import src.preprocessing
importlib.reload(src.preprocessing)
from src.preprocessing import IMG_SIZE, BF_D, BF_SIGMA_COLOR, BF_SIGMA_SPACE, CLAHE_CLIP_LIMIT,
CLAHE_TILE_GRID

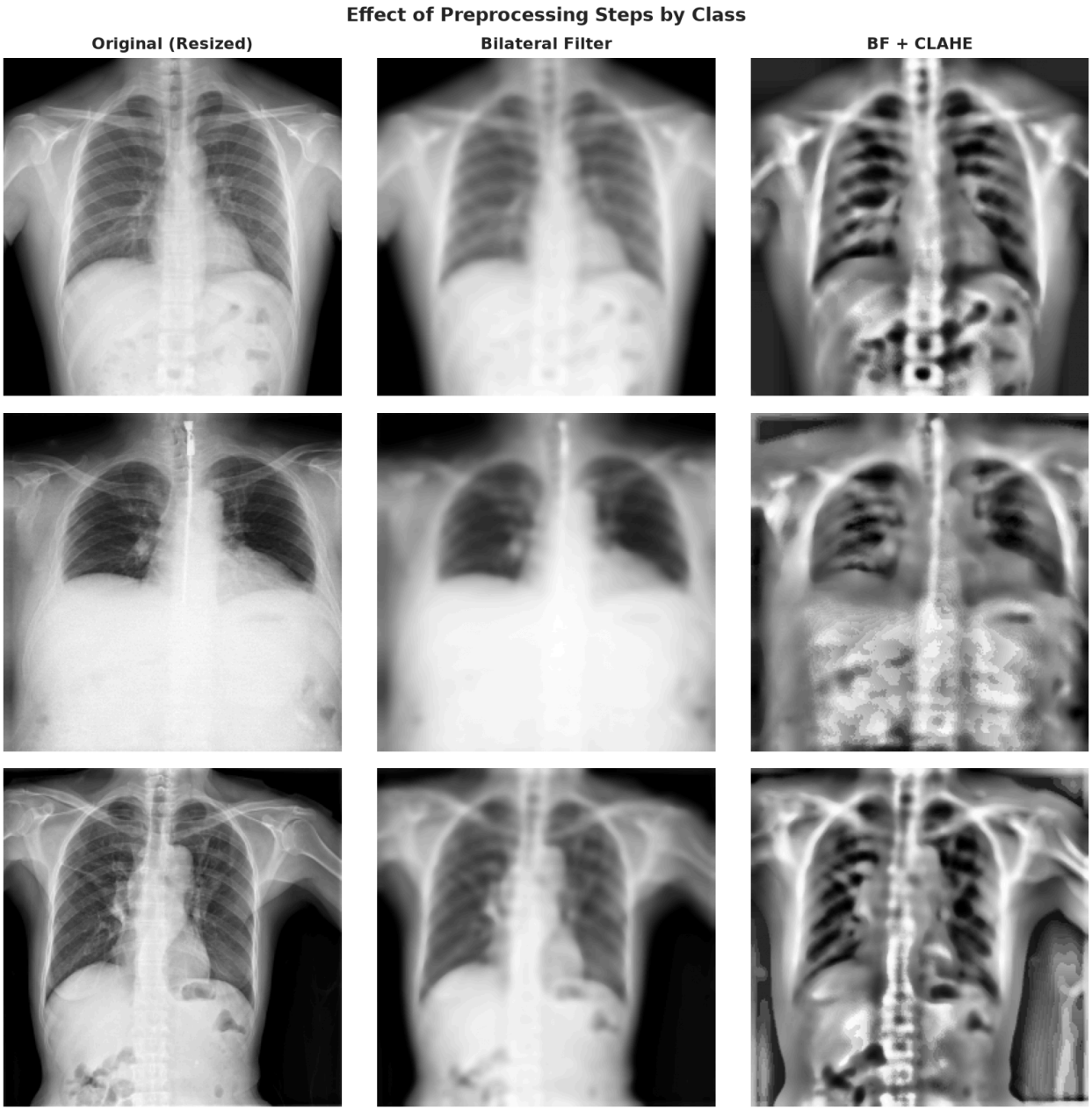
print('Preprocessing pipeline parameters:')
print(f'  Target size   (IMG_SIZE)       : {IMG_SIZE}')
print(f'  Bilateral Filter — diameter    : {BF_D}')
print(f'  Bilateral Filter — sigma color : {BF_SIGMA_COLOR}')
print(f'  Bilateral Filter — sigma space : {BF_SIGMA_SPACE}')
print(f'  CLAHE — clip limit             : {CLAHE_CLIP_LIMIT}')
print(f'  CLAHE — tile grid              : {CLAHE_TILE_GRID}')
```

```
Preprocessing pipeline parameters:
  Target size   (IMG_SIZE)       : (256, 256)
  Bilateral Filter — diameter    : 9
  Bilateral Filter — sigma color : 80.0
  Bilateral Filter — sigma space : 80.0
  CLAHE — clip limit             : 40.0
  CLAHE — tile grid              : (8, 8)
```

3. Comparativa visual: original vs cada paso del pipeline

In [4]:
```
# Se muestra el efecto acumulado de cada paso sobre una imagen representativa
# de cada clase. La visualización en tres columnas permite detectar visualmente
# si algún paso introduce artefactos o degrada la imagen.

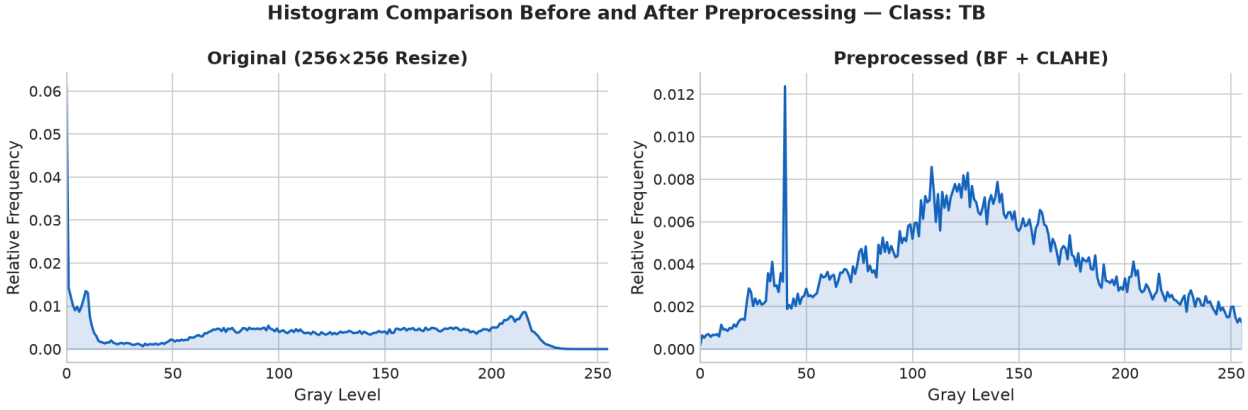
plot_preprocessing_steps(df_base)
```



4. Comparativa de histogramas: antes vs después

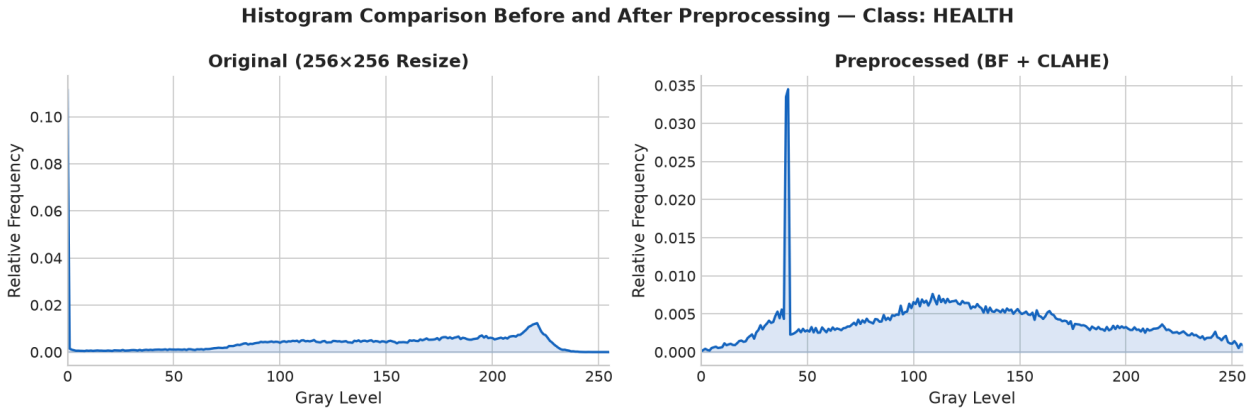
In [5]:
```
# Se comparan los histogramas de intensidad de una imagen de clase `tb`
# antes y después del preprocesamiento completo.
# El CLAHE debe redistribuir la intensidad hacia un histograma más uniforme,
# mejorando el aprovechamiento del rango dinámico.
# Referencia CLAHE: https://docs.opencv.org/4.x/d6/db6/classcv_1_1CLAHE.html

plot_histogram_comparison(df_base, class_name='tb')
```



In [6]:
```
# Se repite la comparativa para la clase `health` para verificar que el
# preprocesamiento tiene un efecto consistente entre clases.

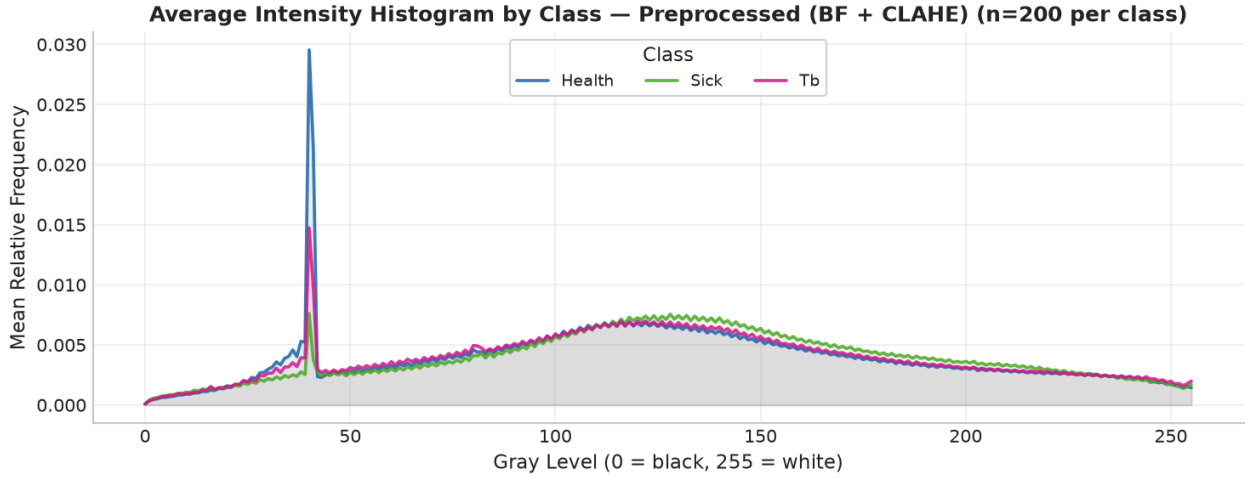
plot_histogram_comparison(df_base, class_name='health')
```



5. Histogramas de intensidad por clase — imágenes preprocesadas

In [7]:
```
# Se replica el análisis del notebook 02 pero sobre imágenes ya preprocesadas.
# La comparación directa entre las dos figuras permite evaluar si el preprocesamiento
# acentúa o reduce las diferencias entre clases.

N_HIST = min(N_INTENSIDAD, len(df_base[df_base['class_name'] == 'health']))
plot_intensity_histograms(df_base, n_hist=N_HIST, preprocessed=True, random_state=42)
```

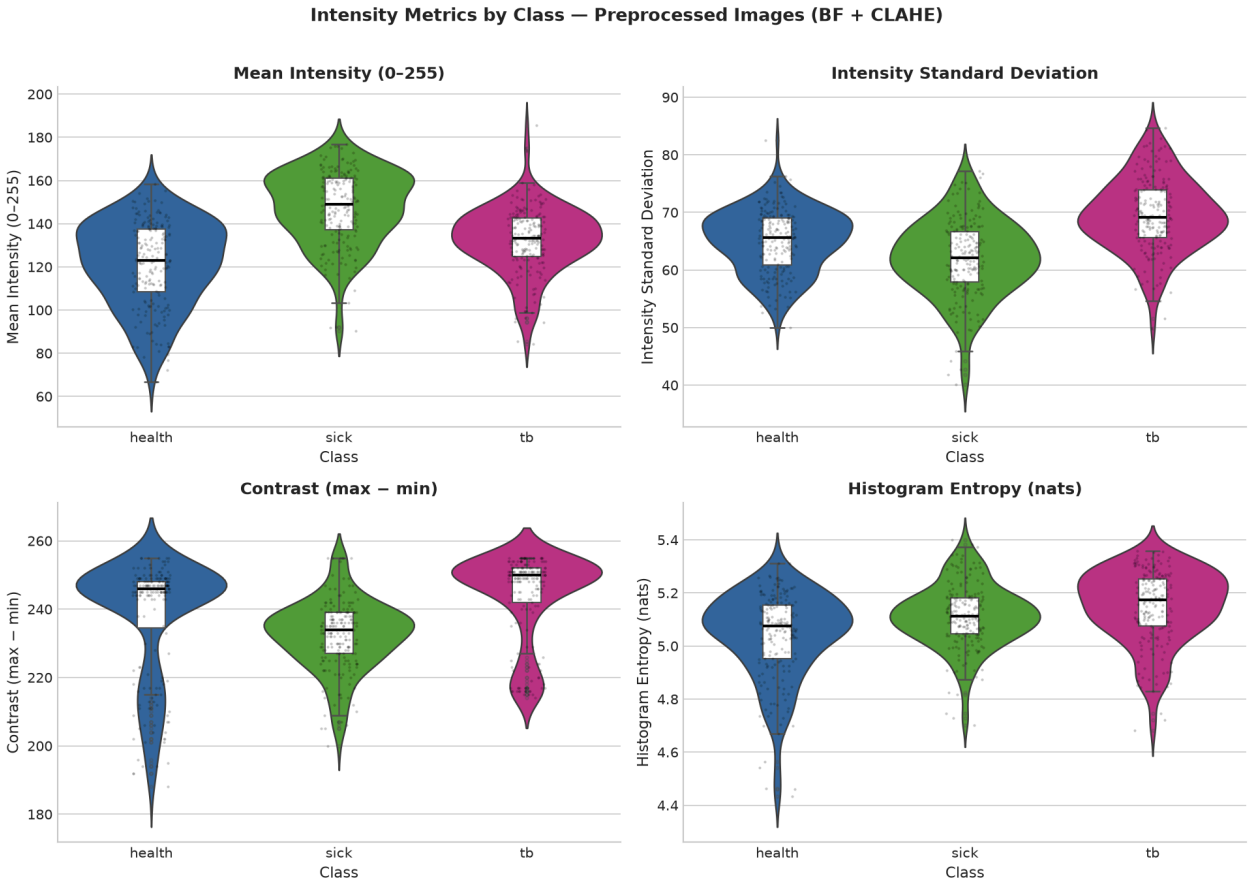


Nota: el pico en niveles de gris bajos (~40) es un artefacto del CLAHE sobre píxeles de fondo (zonas sin tejido). Es esperado en radiografías y no representa información diagnóstica.

6. Distribución de métricas por clase — imágenes preprocesadas

In [8]:
```
# Se calculan y visualizan las métricas de intensidad sobre imágenes preprocesadas.
# La comparación con los resultados del notebook 02 evidencia el efecto del pipeline.
# Nota: compute_intensity_stats trabaja sobre imágenes crudas; para la comparación
# antes/después usar build_preprocessing_stats_table (sección 7).

N_HIST = min(N_INTENSIDAD, len(df_base[df_base['class_name'] == 'health']))
stats_df_prep = compute_intensity_stats(df_base, n_hist=N_HIST, random_state=42)
plot_intensity_metrics(stats_df_prep, title_suffix='Preprocessed Images (BF + CLAHE)')
```



Nota: las formas de los violines dependen del tamaño de muestra definido en src/config.py (N_INTENSIDAD)

7. Tabla de seguimiento: antes y después del preprocesamiento

In [9]:
```python
# Se construye una tabla que consolida las métricas de intensidad promedio
# para cada clase antes y después del preprocesamiento completo.
# Esta tabla es evidencia cuantitativa del efecto del pipeline,
# práctica estándar en artículos de procesamiento de imágenes médicas.

N_SAMPLE = min(N_SEGUIMIENTO, len(df_base[df_base['class_name'] == 'health']))
tabla_seguimiento = build_preprocessing_stats_table(df_base, n_sample=N_SAMPLE, random_state=42)

print(f'Preprocessing tracking table — effect by class (n={N_SAMPLE} per class):')
print()
print(tabla_seguimiento.to_string())
```

```
Preprocessing tracking table — effect by class (n=200 per class):

                     mean    std  contrast  entropy
Class  Stage
health Original      121.21  65.05    238.08   5.0372
       Preprocessed  124.73  60.39    254.91   5.3569
sick   Original      147.36  62.00    232.30   5.1131
       Preprocessed  132.99  57.57    254.94   5.4010
tb     Original      132.26  69.50    244.00   5.1471
       Preprocessed  127.84  59.49    254.94   5.4023
```

Nota: los valores dependen del tamaño de muestra definido en src/config.py (N_SEGUIMIENTO, actualmente 200 por clase). Observaciones esperadas al subirlo:

- contrast → sube a 254-255 en todas las clases tras CLAHE (rango dinámico completo).

- std → baja levemente, el BF reduce ruido local. El contraste de 254.9 ya visible con la muestra actual es consistente con el efecto del CLAHE.

In [10]:
```
# Se calcula el cambio relativo entre el estado original y el preprocesado
# para facilitar la interpretación de los efectos del pipeline.

tabla_pivot = tabla_seguimiento.reset_index()

for metrica in ['mean', 'std', 'contrast', 'entropy']:
    print(f'\nRelative change in {metrica} (Original → Preprocessed):')
    for clase in ['health', 'sick', 'tb']:
        orig = tabla_pivot[(tabla_pivot['Class'] == clase) & (tabla_pivot['Stage'] == 'Original')]
[metrica].values[0]
        prep = tabla_pivot[(tabla_pivot['Class'] == clase) & (tabla_pivot['Stage'] == 'Preprocessed')]
[metrica].values[0]
        cambio = ((prep - orig) / (abs(orig) + 1e-9)) * 100
        print(f'  {clase:>6}: {orig:.2f} → {prep:.2f}  ({cambio:+.1f}%)')
```

```
Relative change in mean (Original → Preprocessed):
  health: 121.21 → 124.73  (+2.9%)
    sick: 147.36 → 132.99  (-9.8%)
      tb: 132.26 → 127.84  (-3.3%)

Relative change in std (Original → Preprocessed):
  health: 65.05 → 60.39  (-7.2%)
    sick: 62.00 → 57.57  (-7.1%)
      tb: 69.50 → 59.49  (-14.4%)

Relative change in contrast (Original → Preprocessed):
  health: 238.08 → 254.91  (+7.1%)
    sick: 232.30 → 254.94  (+9.7%)
      tb: 244.00 → 254.94  (+4.5%)

Relative change in entropy (Original → Preprocessed):
  health: 5.04 → 5.36  (+6.3%)
    sick: 5.11 → 5.40  (+5.6%)
      tb: 5.15 → 5.40  (+5.0%)
```

8. Comparación controlada de preprocesamiento

Se comparan cuatro variantes sobre el mismo subconjunto estratificado (200 imágenes por clase) y las mismas features compactas (intensidad global + GLCM + LBP):

1. `raw`: solo resize.
2. `bilateral`: resize + filtro bilateral.
3. `clahe`: resize + CLAHE.
4. `bilateral+clahe`: cadena actual.

Es una comparación controlada: mismas imágenes, mismas features, distinto preprocesamiento. El objetivo es aportar evidencia sobre la cadena elegida, no afirmar que sea óptima.

In [11]:
```
# Comparación controlada de preprocesamiento sobre un subconjunto.
# Mismas imágenes y mismas features; solo cambia la cadena de preprocesamiento.
# Referencias:
#   preprocesamiento: https://docs.opencv.org/4.x/

import matplotlib.pyplot as plt
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from src.preprocessing import (
    load_grayscale, resize_image, apply_bilateral_filter, apply_clahe,
)
from src.features import feat_intensity_global, feat_glcm, feat_lbp
```

```python
RANDOM_STATE = 42
N_PREPROC = 200
DATA_DIR = Path('..') / 'data'

# Subconjunto estratificado.
partes = []
for clase in df_base['class_name'].unique():
    sub = df_base[df_base['class_name'] == clase]
    partes.append(sub.sample(min(N_PREPROC, len(sub)), random_state=RANDOM_STATE))
muestra = pd.concat(partes).reset_index(drop=True)
print(f'Subconjunto: {len(muestra)} imágenes')
print(muestra['class_name'].value_counts().to_string())

variantes = ['raw', 'bilateral', 'clahe', 'bilateral+clahe']
filas = {v: [] for v in variantes}
y = []

for _, row in muestra.iterrows():
    img = load_grayscale(row['image_path'])
    if img is None:
        continue
    img = resize_image(img)
    imgs = {
        'raw': img,
        'bilateral': apply_bilateral_filter(img),
        'clahe': apply_clahe(img),
        'bilateral+clahe': apply_clahe(apply_bilateral_filter(img)),
    }
    y.append(row['label'])
    for v in variantes:
        f = {}
        f.update(feat_intensity_global(imgs[v]))
        f.update(feat_glcm(imgs[v]))
        f.update(feat_lbp(imgs[v]))
        filas[v].append(f)

y = np.asarray(y)
print(f'Imágenes procesadas: {len(y)}')

cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)
resultados = []

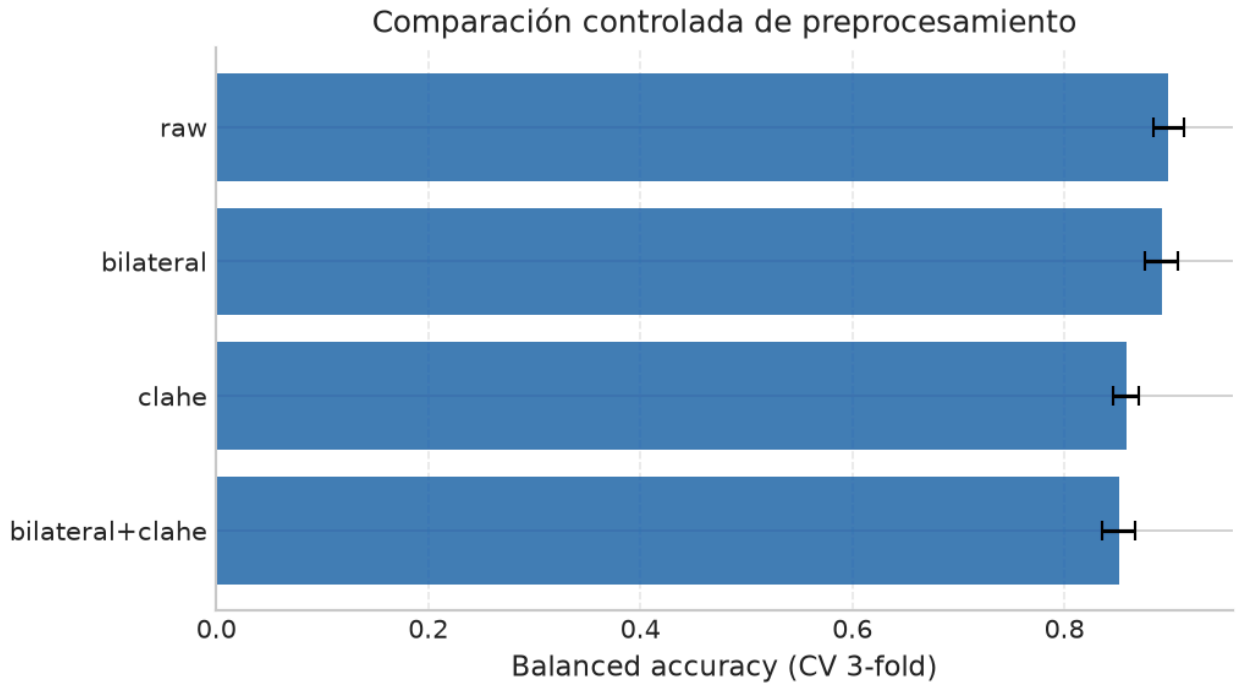
for v in variantes:
    Xv = pd.DataFrame(filas[v]).fillna(0)
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('clf', LogisticRegression(
            class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
        )),
    ])
    bacc = cross_val_score(pipe, Xv, y, cv=cv, scoring='balanced_accuracy', n_jobs=-1)
    f1 = cross_val_score(pipe, Xv, y, cv=cv, scoring='f1_macro', n_jobs=-1)
    resultados.append({
        'Variante': v,
        'BalAcc_media': bacc.mean(),
        'BalAcc_std': bacc.std(),
        'F1_media': f1.mean(),
        'F1_std': f1.std(),
    })

resultados = pd.DataFrame(resultados).sort_values('BalAcc_media',
ascending=False).reset_index(drop=True)
print()
print('Comparación controlada (CV 3-fold, subconjunto):')
print(resultados.round(4).to_string(index=False))
resultados.to_csv(DATA_DIR / '03_comparacion_preprocesamiento.csv', index=False)

fig, ax = plt.subplots(figsize=(7, 4))
orden = resultados.sort_values('BalAcc_media')
ax.barh(orden['Variante'], orden['BalAcc_media'], xerr=orden['BalAcc_std'],
        capsize=4, color='#2166AC', alpha=0.85)
ax.set_xlabel('Balanced accuracy (CV 3-fold)', fontsize=11)
ax.set_title('Comparación controlada de preprocesamiento', fontsize=12)
ax.grid(True, axis='x', linestyle='--', alpha=0.4)
```

```
fig.tight_layout()
plt.show()
```

```
Subconjunto: 600 imágenes
class_name
health    200
sick      200
tb        200
Imágenes procesadas: 600
Comparación controlada (CV 3-fold, subconjunto):
       Variante  BalAcc_media  BalAcc_std  F1_media  F1_std
            raw        0.8982      0.0146    0.8978  0.0149
      bilateral        0.8916      0.0158    0.8908  0.0162
          clahe        0.8583      0.0124    0.8576  0.0121
bilateral+clahe        0.8515      0.0153    0.8505  0.0140
```



Nota: este experimento usa 200 imágenes por clase y features compactas para que sea viable. No sustituye a la extracción completa de features; su función es aportar evidencia controlada sobre el efecto del filtro bilateral y CLAHE. Si una variante no mejora, debe reconocerse en el informe y no atribuir a la cadena actual una superioridad no demostrada.

Conclusiones

Con base en la tabla de seguimiento y las visualizaciones se puede concluir que:

- El Bilateral Filter reduce la desviación estándar (std) en las tres clases. La reducción es de aproximadamente 7.6% en health y sick, y de 14.6% en tb, lo que indica una disminución de la variabilidad de las intensidades después del filtrado.
- El CLAHE aumenta el contraste en las tres clases, llevando los valores medios desde 233.90–243.93 hasta aproximadamente 254.9. Esto indica un aumento del rango dinámico utilizado por las imágenes después del preprocesamiento.
- La entropía aumenta en las tres clases, pasando de valores entre 5.03–5.13 a aproximadamente 5.35–5.40. Esto indica un cambio en la distribución de las intensidades y una mayor diversidad de niveles de gris después del preprocesamiento.
- El efecto del preprocesamiento es consistente entre las tres clases: la desviación estándar disminuye, mientras que el contraste y la entropía aumentan. Sin embargo, estas métricas

descriptivas no permiten determinar por sí solas si el preprocesamiento mejora la capacidad de diferenciar las clases.

Nota: los resultados corresponden a una muestra de 200 imágenes por clase (N_SEGUIMIENTO). Estas observaciones describen el efecto del preprocesamiento sobre esta muestra y deberán complementarse posteriormente con la evaluación de las características y del desempeño de los modelos de clasificación.

Siguiente etapa: `04_Analisis_Regional.ipynb`